Solution: Take Gifts From the Richest Pile(easy)

Problem Statement

You're presented with several piles of gifts, with each pile containing a certain number of gifts. Every second, you'll engage in the following activity:

- Pick the pile that contains the highest number of gifts. If multiple piles share this distinction, you can select any of them.
- Compute the square root of the number of gifts in the selected pile, and then leave behind that many gifts (rounded down). Take all the other gifts from this pile.
- You'll do this for "k" seconds. The objective is to find out how many gifts would still remain after these "k" seconds.

Examples

1.
    - **Input**: gifts = [4, 9, 16], k = 2
    - **Expected Output**: 11
    - **Justification**:
        1. Take from third pile (16 gifts): leave ($\sqrt{16}$) = 4 gifts, take 12. Remaining gifts = [4, 9, 4]
        2. Take from second pile (9 gifts): leave ($\sqrt{9}$) = 3 gifts, take 6. Remaining gifts = [4, 3, 4]

2.
    - **Input**: gifts = [1, 2, 3], k = 1
    - **Expected Output**: 4
    - **Justification**:
        1. Take from third pile (3 gifts): leave ($\sqrt{3}$) = 1 gift (rounded down), take 2. Remaining gifts = [1, 2, 1]

3.
    - **Input**: gifts = [25, 36, 49], k = 3
    - **Expected Output**: 18
    - **Justification**:
        1. Take from third pile (49 gifts): leave ($\sqrt{49}$) = 7 gifts, take 42. Remaining gifts = [25, 36, 7]
        2. Take from second pile (36 gifts): leave ($\sqrt{36}$) = 6 gifts, take 30. Remaining gifts = [25, 6, 7]
        3. Take from first pile (25 gifts): leave ($\sqrt{25}$) = 5 gifts, take 20. Remaining gifts = [5, 6, 7]

Solution

In order to solve this problem, we leverage a max heap data structure. A max heap allows us to efficiently retrieve and process the pile with the most gifts at any given time. Once we choose the pile with the maximum number of gifts, we compute the number of gifts we'd leave behind (which is the floor value of the square root of the gifts in that pile) and then push this new value back into the max heap. This operation is repeated 'k' times. After these 'k' operations, we sum up the remaining gifts in all the piles to get our final answer.

Detailed Explanation:

1. **Heap Initialization**: We start by creating a max heap. However, some programming languages, like Python, provide a min heap by default. To turn this into a max heap, we can insert negative values. By doing so, retrieving the smallest element from this heap effectively gives us the pile with the most gifts.

2. **Processing 'k' Operations**: For each of the 'k' operations, we:

    - Pop the heap to get the pile with the most gifts.
    - Calculate the gifts we'll take by subtracting the floor value of the square root of the current pile size.
    - Push the remaining gifts back into the max heap.

3. **Calculating Remaining Gifts**: After the 'k' operations are completed, we simply sum up all the elements present in our heap. This total represents the number of gifts left across all piles.

4. **Returning the Result**: Finally, the accumulated result is returned as the answer.

This algorithm efficiently processes the piles and allows us to always operate on the pile with the most gifts, ensuring that the gifts are taken in the most optimal manner as per the problem's requirements. The use of a max heap provides a clear way to consistently choose the correct pile to operate on without having to sort or iterate through the entire array in each of the 'k' operations.

Algorithm Walkthrough:

Using the input gifts = [4, 9, 16], k = 2:

- Convert gifts to a max-heap: [16, 9, 4]
- 1st second:
    - Pop 16. Calculate ($\sqrt{16}$) = 4. Push back 4 in the max-heap.
    - New heap: [9, 4, 4]

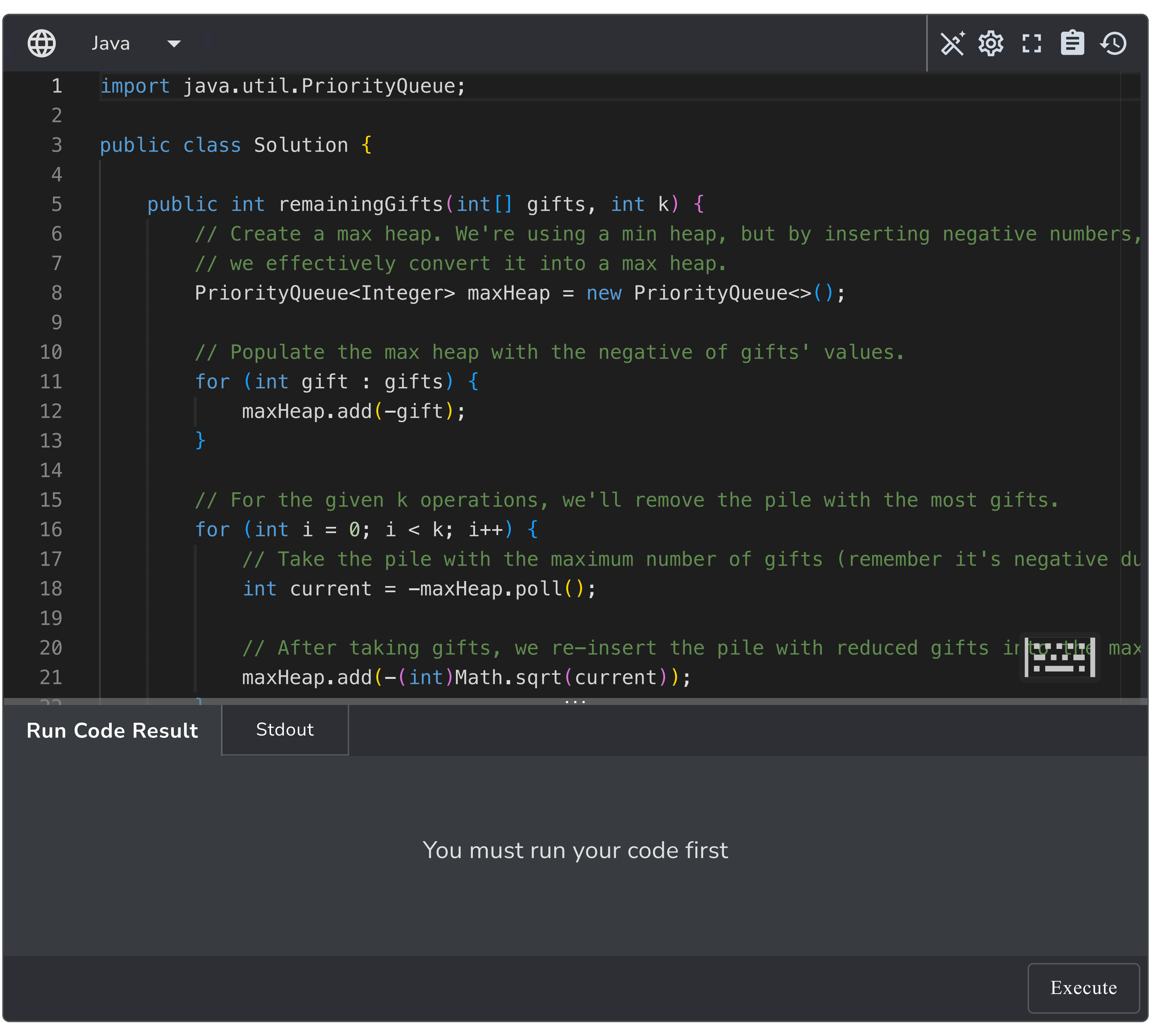
- 2nd second:
    - Pop 9. Calculate ($\sqrt{9}$) = 3. Push back 3 to the max-heap.
    - New heap: [4, 4, 3]

- Remaining gifts in the heap: [4, 4, 3]

Thus, the total remaining gifts = 4 + 4 + 3 = 11.

Code

Here is the code for this algorithm:



```java
import java.util.PriorityQueue;

public class Solution {

    public int remainingGifts(int[] gifts, int k) {
        // Create a max heap. We're using a min heap, but by inserting negative numbers,
        // we effectively convert it into a max heap.
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>();

        // Populate the max heap with the negative of gifts' values.
        for (int gift : gifts) {
            maxHeap.add(-gift);
        }

        // For the given k operations, we'll remove the pile with the most gifts.
        for (int i = 0; i < k; i++) {
            // Take the pile with the maximum number of gifts (remember it's negative du
            int current = -maxHeap.poll();

            // After taking gifts, we re-insert the pile with reduced gifts into the max
            maxHeap.add(-(int)Math.sqrt(current));
```

You must run your code first

Time Complexity:

1. **Heapification:** Converting the given `gifts` list to a heap is `O(n*log(n))` where `n` is the length of the gifts array. Although in a strict sense, building a heap from an unordered array is `O(n)`, in our solution we perform an `O(n*log(n))` operation since we're continuously pushing items to the heap.

2. **K Operations**: For each of the `k` operations, we:

    - Pop an item from the heap, which is `O(log(n))`.
    - Push the reduced item back to the heap, which is again `O(log(n))`.

    Thus, for `k` operations, this part of the algorithm runs in `O(k*log(n))`.

3. **Final Summation**: After the `k` operations, summing up the remaining gifts in the heap takes linear time `O(n)`.

Combining these, our total time complexity is: [$O(nlog(n) + klog(n) + n)$] For large values of `n` and `k`, this approximates to: [$O(nlog(n) + klog(n))$]

Space Complexity:

1. **Heap Space**: We use a max-heap to store the gifts which takes `O(n)` space.

2. **Constant Space:** Other variables and counters used in the algorithm take constant space, `O(1)`.

Hence, the total space complexity is: [$O(n)$]

← Previous
Take Gifts From The Richest Pile(Easy)
Next →
Sort Characters By Frequency (Easy)
☐ Mark as Completed